```
logUserDataFor 'fred'
# 'An error occurred'
```

Ideally you'd have a database that doesn't suffer frequent timeouts, but suppose the database can't be replaced. The only way to fix the problem is to change the program so that if the database request fails, it will retry until it gets a response. How do you do this? The first retry is easy to write; just put it directly in the callback:

```
logUserDataFor = (user) ->
  users.get user, (error, data) ->
    if error then users.get user, (error, data) ->
      if error then console.log 'An error occurred both times'
      else 'Got the data (on the second attempt)'
    else console.log 'Got the data'
```

How about the second retry or the third, fourth, or tenth retry? You certainly don't want to nest 10 retries (that would be so horrendous that it's not even shown here). Take a look at the alternative:

```
logUserDataFor = (user) ->
  dbRequest = ->
    users.get user, (error, data) ->
      if error then dbRequest()
      else console.log 'Got the data'
  dbRequest()
```

dbRequest is invoked by a callback defined inside dbRequest.

If the database finally responds the fourth time it's called, you'll see that logged:

```
logUserDataFor 'fred'
# 'An error occurred'
# 'An error occurred'
# 'An error occurred'
# 'Got the data'
```

It's called *recursion* when a function invokes itself, either directly or indirectly. Although the recursive `logUserDataFor` function works, it's a bit heavy-handed. Instead of trying the request again until the end of time (or the program is terminated), retry a failed database request once every second for the next 5 seconds before giving up: